

# Assembler

Комбинирование разноязыковых модулей

Подготовил  
Старший преподаватель  
МИЭМ НИУ ВШЭ  
Морозов Владимир Игоревич

# Основные положения

- Т.к. ассемблер является языком, наиболее приближенным к аппаратной части компьютера, его инструкции выполняются очень быстро
- Эта особенность может быть полезна при программировании ресурсоёмких вычислений с данными
- Однако те части программы, которые не относятся к вычислениям, неудобно реализовывать на ассемблере (например, интерфейс пользователя). В таком случае возникает необходимость использовать ассемблер для вычислительных модулей программы, и более высокоуровневый язык для остальных.

# Модули на ассемблере

- Под модулем, написанным на ассемблере, в данном случае понимается отдельный файл, в котором описаны реализации некоторых функций
- Эти функции могут быть вызваны из программы на другом, более высокоуровневом, языке программирования (в нашем случае – Си)

Реализация для ассемблера AT&T

# Объявление функции

- Чтобы использовать в программе на Си функцию, написанную на ассемблере, прежде всего необходимо объявить её с ключевым словом **extern**
- В ассемблерном файле под реализацией функции понимается подпрограмма, отмеченная отдельной меткой
- В файле на ассемблере следует указать, что функция глобальная с помощью **.globl**
- Далее функцию следует использовать в программе на Си, как и любую другую обычную функцию

# Передача аргументов

- Аргументы в функции на ассемблере на UNIX-подобных системах передаются через регистры
- Используется следующий набор регистров: `rdi`, `rsi`, `rdx`, `rcx`, `r8`, `r9`
- Аргументы, переданные функции, помещаются по порядку в регистры из данного перечня. Например, при вызове функции  
`int f(int x, int y)`  
значения аргументов `x` и `y` будут помещены в регистры `edi` и `esi` (версии `rdi` и `rsi` соответствующей разрядности).

# Передача аргументов

- Все регистры, начинающиеся на r, занимают по восемь байт
- Если тип аргумента занимает меньший размер, аргумент будет помещён в младшую часть соответствующего регистра
- **Все указатели занимают 8 байт** по умолчанию (если не указано, что сборка под x86)

# Возврат значения из функции

- Чтобы вернуть значение из функции, необходимо поместить его в регистр `rax` (или аналогичный соответствующей разрядности, например, `eax` или `ax`)



# Безопасность операций

- Программный модуль на Си при компиляции преобразуется в ассемблерный код, а значит, программа на Си тоже использует регистры и хранит в них какие-то важные значения
- Программа на Си “не знает” о действиях, производимых в модуле на ассемблере, а значит, любые изменения в регистрах, появившиеся после вызова подпрограммы на ассемблере (за исключением регистра `rax`), могут негативно повлиять на работу основной программы на Си
- Следовательно, **содержимое регистров** после вызова подпрограммы на ассемблере должно остаться **в неизменном виде**

# Работа со стеком

- В ассемблере существует возможность работы со структурой данных типа “стек” (LIFO)
- Для этого используются команды:
  - `push` – принимает единственный аргумент и помещает его в стек
  - `pop` – принимает единственный аргумент и помещает в него содержимое вершины стека
- **Ячейки стека имеют размер 8 байт**

# Безопасность операций

- Классическим подходом для обеспечения неизменности регистров в ходе выполнения подпрограммы на ассемблере является сохранение значений всех регистров в стек перед началом работы подпрограммы
- В конце работы подпрограммы все значения должны быть извлечены из стека и помещены обратно в те же регистры, где были изначально
- В стек необходимо сохранять все используемые в ходе работы подпрограммы регистры, **в том числе и используемые неявно**

# Ассемблерные вставки

# Общие положения

- В программах на Си существует возможность непосредственной вставки ассемблерного кода
- Для этого при использовании компилятора gcc используется инструкция asm
- После ввода данной инструкции в скобках **через двоеточие** вводится следующая информация:
  - Ассемблерный код
  - Список выходных операндов
  - Список входных операндов
  - Список разрушаемых регистров

# Описание операндов

- Списки входных и выходных операндов используются для автоматического копирования переменных из кода на Си в регистры для работы с ними в ассемблере
- Списки оформляются следующим образом:  
"код\_регистра" (название\_переменной)
- Если переменных несколько, они перечисляются через запятую
- Код регистра означает, в какой регистр будет скопировано значение переменной. Если в качестве кода поставить `x` регистр будет выбран автоматически и будет доступен в ассемблерном коде по номеру в порядке объявления (например, `%0` или `%1`)

# Описание операндов

- Перед кодом выходного операнда ставится символ =

Коды регистров

| Символьный код | Регистры    |
|----------------|-------------|
| a              | eax, ax, al |
| b              | ebx, bx, bl |
| c              | ecx, cx, cl |
| d              | edx, dx, dl |
| S              | esi, si     |
| D              | edi, di     |

# Оформление кода во вставке

- Вставка может быть представлена несколькими строковыми константами, не разделёнными никакими символами
- В конце каждой строки должны стоять символы "`\n\t`"
- Сохранение регистров выполнять в коде не нужно, это выполняется автоматически



# Сохранение значений регистров

- Чтобы значения регистров были сохранены, их имена нужно перечислить в кавычках с префиксом % после последнего двоеточия в инструкции вставки

# Пример кода со вставкой

```
int f1(int x, int y)
{
    int res = 0;

    __asm__ (
        "addl %1, %2\n\t"
        "movl %2, %0\n\t" :
        "=r" (res) :
        "r" (x), "r" (y)
    );

    return res;
}
```

Компиляция, запуск и отладка

# Инструменты компиляции и отладки

- Для компиляции программы, включающей модуль на языке ассемблер, используется тот же компилятор gcc. Для этого его нужно вызвать следующим образом:

```
gcc имя_файла_си.с имя_asm_файла.s -o имя_исп_файла -g
```

- Отладка производится аналогично отладке программ на ассемблере в помощью gdb
- При выполнении шага со входом в ассемблерную функцию отладчик автоматически перейдёт к ассемблерному коду

# Полезные источники

- <http://asmcourse.cs.msu.ru/wp-content/uploads/2013/04/gcc-inline-asm.pdf> –  
подробнее об ассемблерных вставках